

Recursive Self-Improvement in AI: A Critical Survey of Mechanisms, Evidence, and Evaluation

Working draft | Literature cutoff: 6 September 2026

Abstract

Recursive self-improvement (RSI) concerns artificial intelligence systems that improve the mechanisms responsible for their subsequent improvement. The term now covers heterogeneous practices, including output revision, persistent memory, synthetic-data training, self-play, agent-code evolution, and automated research. These practices differ in what changes, which signals justify change, and whether an improved system actually becomes a more effective improver. This critical narrative survey synthesizes 53 selected references spanning theoretical foundations and contemporary language-model and agent systems, with a literature cutoff of 6 September 2026. We distinguish transient refinement, persistent adaptation, and improvement of an improvement operator; relate these distinctions to existing surveys; and examine how recent systems connect their operational and developmental loops. Particular attention is given to self-modifying coding agents, self-directed weight adaptation, and emerging benchmarks for persistent experience, data-centric research, and training-algorithm design. Our assessment of the selected literature supports bounded, heterogeneous forms of self-improvement, while leaving general, sustained, resource-normalized recursive acceleration unestablished. Building on prior work on metaproductivity and controlled agent evaluation, we synthesize a proposed evaluation protocol based on matched starting systems, frozen-operator controls, isolated audit feedback, lineage records, and complete resource accounting. The protocol is a methodological proposal, not an experimentally validated contribution. We conclude with research priorities for measuring inherited improvement capacity, maintaining evaluator integrity, and testing transfer across tasks, models, and generations.

1 Introduction

An AI system can produce a better answer without becoming a better system. It can become a better system without improving the procedure that produced the change. It can improve that procedure without sustaining an accelerating sequence of further improvements. These distinctions are central to recursive self-improvement, but are easily obscured when all three phenomena are described as systems that “learn by themselves.”

Good’s discussion of an ultraintelligent machine made machine design an important part of the intelligence-explosion argument [1]. Later, the Gödel machine gave self-modification a formal decision-theoretic treatment: software rewrites are justified through proofs about expected utility relative to a specified formal system [2]. Contemporary research operationalizes parts of this ambition through executable programs, foundation models, training pipelines, and empirical evaluators. The engineering questions are now concrete: what may change, who proposes a change, what feedback selects it, and what does the next generation inherit?

The resulting literature is neither a single algorithmic family nor a uniform progression toward

autonomy. A language model may generate rationales used in subsequent training; an agent may preserve debugging lessons in memory; a coding system may alter the tools through which it edits its own implementation; and an automated researcher may design an improved training procedure for a separate target model. Each can yield useful gains. Whether it provides evidence of recursion depends on the system boundary and on how the gain influences the next improvement cycle.

Several substantial surveys already organize this field. Gao et al. classify self-evolving agents by the objects, timing, mechanisms, and settings of evolution; Fang et al. provide a framework connecting inputs, agents, environments, and optimizers [3, 4]. Chen et al. distinguish bounded refinement from autonomous research loops and emphasize evaluative signals; Ren et al. model persistent updates across foundation models and agent scaffolds [5, 6]. Liu et al. explicitly include the improvement mechanism in an evolving system and propose capability grades, while Zhou et al. focus on software-specific evolution [7, 8]. Consequently, another taxonomy alone is insufficient grounds for a strong novelty claim.

This survey instead centers on the evidential gap between **a system that improves** and **a system that improves its capacity to improve**. This gap is also not newly discovered here. Huxley–Gödel Machine (HGM) explicitly studies the mismatch between benchmark performance and metaproductivity, or the ability to produce productive descendants [9]. Our contribution is a critical synthesis connecting that distinction to persistent-state controls, automated-training benchmarks, adaptive evaluation, and resource accounting. The resulting evaluation proposal is intended to make RSI claims more comparable; its usefulness still requires empirical validation.

Three questions guide the review. First, which mechanisms persistently alter the system, and which merely reorganize computation on the current task? Second, what evidence connects an inherited modification to better subsequent improvement? Third, what additional observations would justify stronger claims about transfer, repeated recursion, or acceleration? A clear answer to these questions is more informative than a binary declaration that RSI has or has not arrived.

2 Review scope and method

This is a **critical narrative survey**, not an exhaustive systematic review or a meta-analysis. The evidence base contains 53 selected publication records. Searches were conducted on 6 September 2026 using combinations of “recursive self-improvement,” “self-evolving agents,” “self-modifying coding agents,” “self-edit,” “metaproductivity,” and “AI research benchmark,” followed by targeted searches for named methods and references in recent surveys. Primary records were checked on arXiv, publisher pages, and author-maintained project pages. arXiv metadata were also retrieved directly to verify titles, author lists, first-posting dates, and consulted versions.

Works were selected for one of four purposes: theoretical foundations; representative mechanisms that change answers, persistent state, or improvement procedures; empirical evaluations relevant to recursion; or evidence about feedback failures and statistical validity. Recent surveys were included to establish overlap and avoid unsupported priority claims. Marketing articles, informal social-media claims, papers about the financial relative strength index, and general AI applications without a relevant improvement mechanism were excluded from the evidential core.

Reading depth was selective. Abstracts and bibliographic records were checked across the reference set; methods, evaluation details, or relevant limitations were examined for the central case studies. The companion source register records the consultation basis, and the evidence ledger identifies the sections supporting quantitative statements. This workflow does not establish that all relevant

literature was found or that every appendix of every cited paper was inspected. Bibliographic verification, close reading, and independent experimental reproduction are distinct activities; no reproduction experiments were performed for this survey.

The cutoff is a search boundary, not a claim of complete coverage through that date. Several July–August 2026 studies are recent preprints. Their results are treated as author reports rather than independently established facts. We preserve benchmark subsets, evaluator access, and budget qualifications when discussing numerical results. Heterogeneous scores are not pooled into a single ranking or used to estimate an overall effect size. Classifications below are this survey’s interpretations of the described mechanisms, not claims that the original authors adopt our terminology.

3 What counts as recursive self-improvement?

3.1 A system boundary and two coupled loops

Let an AI system at generation t be represented by

$$S_t = (\theta_t, h_t, m_t, d_t, u_t, v_t),$$

where θ_t denotes model parameters, h_t the operational scaffold, m_t persistent memory and skills, d_t retained training resources, u_t the improvement procedure, and v_t the internal evaluation or selection procedure. A resource allocation b_t and a record of experience e_t condition the next update:

$$S_{t+1} = \text{Commit}(S_t, u_t(S_t, e_t; b_t), v_t).$$

This notation is a bookkeeping device, not a claim that the components are independent. The same model can implement both task behavior and the improver; memory can contain executable tools; and a trainer may be represented partly in code and partly in generated instructions. Earlier system-level surveys already motivate coupled representations of this kind [6, 7].

The **operational loop** executes tasks and produces outcomes. The **developmental loop** converts evidence from those outcomes into candidate changes, evaluates them, and commits selected changes. Recursion concerns an inherited change that affects the developmental loop itself. Figure 1 illustrates why a path from outcomes to updates is necessary but insufficient: one must also establish what was inherited and how it affected later improvement.

An audit evaluator, denoted V^* , is deliberately outside the modifiable state in the evaluation protocol proposed later. This is an experimental boundary, not a metaphysical claim that a real system has an immutable external judge. Internal evaluators may evolve, but their progress must be assessed against evidence that the candidate cannot freely redefine.

3.2 Three distinctions and a stronger hypothesis

Transient refinement changes an answer, plan, or working context during an episode without preserving the change for later episodes. **Persistent self-improvement** commits a model, memory, tool, prompt, or code update induced by the system’s experience. **Recursive improvement** further changes the inherited mechanism used to generate or select subsequent improvements. These categories characterize mechanisms; they are not an ordinal safety or intelligence scale.

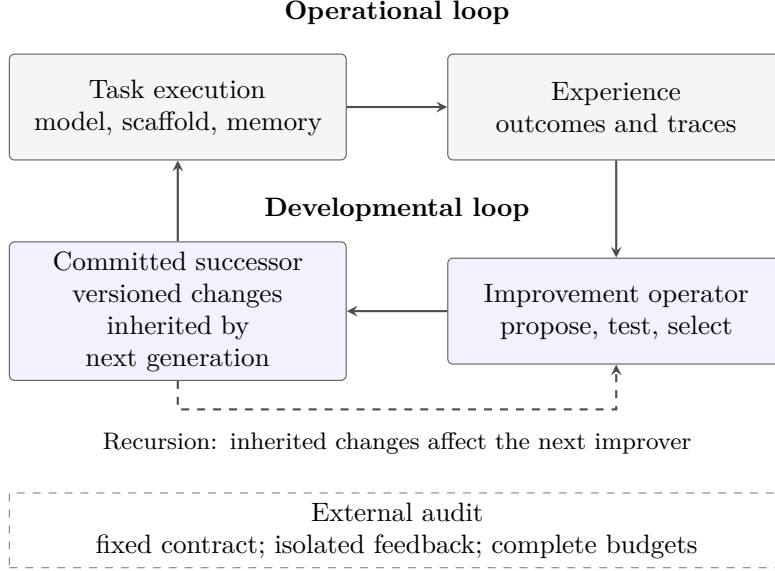


Figure 1: Two coupled loops and the additional inheritance relation needed for a recursive claim. The external audit is an experimental boundary in the proposed evaluation protocol. This diagram is the survey’s synthesis, not a reproduced figure.

An additional claim, **sustained recursive acceleration**, concerns the dynamics of a sequence of improvements. It requires evidence that the capacity to produce useful changes increases over repeated generations under an explicit treatment of resources and task difficulty. It does not follow from self-reference alone. A self-editing program can plateau, oscillate, become more expensive, or destroy capabilities that matter outside its selection benchmark.

Weight changes are neither necessary nor sufficient for the operational definition used here. A frozen foundation model combined with a changing code-based improver can exhibit bounded recursion at the compound-system level. Conversely, a neural network trained repeatedly by a fixed optimizer exhibits learning without demonstrating that the optimizer or the overall improvement mechanism became more effective. STOP uses a stricter terminology and explicitly avoids claiming full RSI because its underlying language model remains fixed [10]. Reporting the boundary resolves much of this apparent disagreement.

Likewise, a system that trains another model is not automatically improving itself. A successor relation must be specified: does the resulting model, optimizer, or scaffold become part of the next research agent? Without this inheritance, the result supports automated AI development, which is relevant to RSI but does not establish a closed recursive lineage.

3.3 Mechanisms and evidence should be reported separately

A useful description records both the proposed modification and the evidence supporting its effect. We use four evidential questions: does a benefit survive a fresh session; does it survive a matched resource comparison; does the updated improver outperform a frozen improver from comparable starting conditions; and does that advantage survive further generations or distribution shifts? These are separable questions rather than mandatory levels of one ladder.

For example, transferring an evolved scaffold to another model supports portability of the scaffold.

It does not by itself show that the transplanted system is better at producing its next modification. Similarly, a successful code rewrite demonstrates execution and persistence; it does not establish that the rewrite caused a gain rather than merely accompanying additional search. This separation prevents architectural access from being mistaken for demonstrated capability.

4 Mechanistic lineages

4.1 Formal self-modification, meta-learning, and open-ended search

The Gödel machine is a conceptual anchor because it makes improvement of the system’s own software part of the optimization problem. Its guarantees are conditional on the axioms, utility specification, and availability of suitable proofs. They do not guarantee that useful proofs will be found cheaply or that a specified utility captures all real-world interests [2]. An empirical coding agent inspired by this idea does not inherit the theorem merely by evaluating a patch with tests.

Learned optimizers provide a different route to improving improvement procedures. Andrychowicz et al. learn optimization behavior from a distribution of training problems, while AutoML-Zero searches over low-level programmatic building blocks for learning algorithms [11, 12]. These methods can improve a learner’s update rule without requiring that the learner autonomously redesign the entire outer search process. They also highlight an older and still relevant issue: improvement is measured relative to a chosen task distribution and search representation.

Self-play and environment co-evolution contribute mechanisms for producing changing learning challenges. AlphaZero’s predecessor formulation learns chess and shogi behavior through self-play under a fixed game specification and training procedure [13]. POET jointly develops environments and their solutions, with transfer between evolving tasks [14]. Neither changing opponents nor increasingly difficult environments alone establishes recursive algorithm redesign. Their relevance is that useful learning opportunities can be generated by the system rather than supplied as a static list.

4.2 Output revision, memory, and reusable tools

Self-Refine uses feedback on a generated output to guide another revision; Reflexion stores verbal feedback that can influence subsequent attempts; Voyager accumulates an executable skill library during exploration [15–17]. They illustrate distinct persistence boundaries. Output refinement can remain episode-local, whereas retained feedback and skills can change later behavior without updating model weights. Toolformer adds a parametric route by training a model on automatically selected tool-use examples [18].

These methods matter to RSI because a reliable improver needs memory, tools, and error correction. However, adding an experience to memory is only a candidate mechanism. A fresh-session comparison must show that it is retrieved and used appropriately, and that stale or irrelevant experience does not impair later tasks. The stronger question is whether the system improves how it writes, selects, retrieves, or revises that memory.

Intrinsic self-correction also has a contested empirical record. Huang et al. find failures of reasoning self-correction without external feedback in their studied settings, whereas subsequent work reports that prompt fairness and decoding choices materially affect the outcome [19, 20]. SCoRe trains multi-turn self-correction through reinforcement learning and reports improvements after targeted

training [21]. These findings concern different interventions. They do not warrant either a universal impossibility claim or an assumption that repeated untrained self-critique is reliably beneficial.

4.3 Synthetic supervision and co-evolving curricula

STaR bootstraps reasoning traces into further training, while Self-Instruct generates and filters instruction-following examples [22, 23]. Such pipelines can consolidate useful behavior into parameters. They do not create information from nothing: pretrained knowledge, seed examples, filtering rules, known answers, and environmental regularities can all contribute supervision. A training dataset’s origin should therefore be described component by component rather than compressed into “self-generated.”

Self-Rewarding Language Models couples response generation and model-produced evaluative feedback across iterative preference optimization. SPIN instead learns through a self-play objective anchored in a supervised dataset [24, 25]. Their feedback channels differ. A model-generated preference is a learned judgment; a supervised reference is an external anchor relative to the generated alternative. Neither channel should automatically be treated as error-free.

Reinforcement learning with verifiable rewards offers another path. DeepSeek-R1 demonstrates that reinforcement learning can substantially shape reasoning behavior, but its training procedures are not thereby autonomously self-redesigned [26]. Absolute Zero Reasoner (AZR) couples task proposal and solving through executable code-based feedback [27]. R-Zero co-evolves challenger and solver models and derives pseudo-labels through the solver’s answer agreement [28]. Executable verification and majority agreement have different error properties: a consensus answer can be systematically wrong, while a passing execution check can still cover only a limited specification.

“Zero data” in these studies refers to a particular post-training input regime. It does not mean absence of pretraining, human-authored software, chosen objectives, or environmental structure. Their main importance is to redistribute curriculum and supervision work from a fixed human-curated post-training dataset into an automated loop. Whether that loop improves its own learning mechanism remains a separate empirical question.

4.4 Evolving prompts, scaffolds, and improvers

Automated Design of Agentic Systems (ADAS) uses a meta-agent to program candidate agents and maintains an archive of prior designs. GEPA uses natural-language reflection and evolutionary selection to optimize prompts [29, 30]. Both make substantial parts of agent construction searchable. Yet optimizing a prompt or an agent architecture with a fixed search controller is different from modifying the controller that performs the search.

STOP directly applies an improver program to itself. The resulting program changes how language-model calls are organized to improve downstream programs [10]. Gödel Agent likewise exposes the agent’s logic to self-directed modification [31]. These systems move the object of search toward the developmental mechanism. The remaining external structure, including objectives, runtime limits, model APIs, and evaluation procedures, still matters when interpreting how much of the loop has closed.

SICA and Darwin Gödel Machine (DGM) make self-modification concrete in coding agents. DGM retains a population archive that allows previously discovered agents to seed further changes; SICA optimizes an agent through its own coding capabilities [32, 33]. Their results support the value of inherited scaffolds, but the empirical interpretation depends on subsets, budgets, selection, and

controls. In DGM’s consulted version, archive maintenance and parent selection remain fixed. The boundary therefore includes an evolving self-modifying agent within an externally specified evolutionary process.

HGM targets a weakness of selecting parents by present task performance. It estimates clade-level metaproductivity from descendant outcomes and separates expansion, evaluation, and selection decisions [9]. This is directly relevant to the question “does the improver improve?” However, a proposed outer search strategy should not automatically be described as autonomously discovered. Its evidential contribution concerns lineage-sensitive search and reported comparisons, not elimination of all human-designed improvement machinery.

4.5 Self-directed weight adaptation

SEAL trains a model to generate self-edits: training data and update directives whose utility is assessed after adaptation. Its outer reinforcement-learning loop rewards edits that improve the adapted model’s downstream performance [34]. This gives the language model a role in choosing how new information becomes a parameter update. It also illustrates why persistence requires careful qualification: an adaptation episode, a deployed accumulated model, and an outer-loop training run are different objects.

Search over Self-Edit Strategies studies a narrower extension in which the model searches over adaptation templates and selected hyperparameters. Its reported setup uses one round of self-improvement; an archive helps relative to a weaker template but does not surpass the strongest human-designed baseline, and homogenization remains a concern [35]. This is useful negative or limiting evidence. Enlarging a modifiable interface does not guarantee that the model exploits the added freedom productively.

A full evaluation should distinguish improvement of edit proposals from improvement caused by a stronger starting model or a different training budget. It should also measure whether previous adaptations are retained. A system that incorporates new information efficiently while repeatedly damaging old capabilities has not demonstrated an unrestricted cumulative learning process.

4.6 Algorithm discovery and automated research

FunSearch combines pretrained language-model proposals, program evaluation, and a population of candidates to discover useful programs. AlphaEvolve extends evolutionary coding to a broader set of algorithmic and infrastructure problems [36, 37]. These systems show why external execution can make a generative model useful beyond its unaided responses. Improving infrastructure used in model development creates a potentially important feedback path, but that path should be measured end to end before it is called autonomous recursive acceleration.

The AI Scientist and its successor automate portions of hypothesis generation, experimentation, analysis, and manuscript preparation [38, 39]. A completed paper or successful experiment is evidence of research automation. For RSI, the critical additional step is inheritance: does the discovery improve the researcher that will conduct the next research cycle? Scientific output volume, reviewer scores, and successor-model capability are not interchangeable measures.

The field therefore contains several partially connected loops. An improved coding scaffold can build a better data generator; the generator can improve a model; the model can propose better scaffold changes. Such coupling is plausible, but a diagram of the coupling is not evidence that its net gain is positive. Interface costs, transfer losses, evaluation noise, and model-training expense may dominate

the apparent local improvements.

5 Comparative evidence from representative systems

Table 1 summarizes mechanism boundaries. “Recursive relevance” denotes our interpretation of the described design, not an independently verified grade.

Table 1: Representative mechanisms and the boundary of recursive claims.

Family	Persistent object	Main feedback	Recursive relevance and boundary
Self-Refine	Usually episode-local output	Model critique	Refinement unless changes are retained.
Reflexion / Voyager	Memory / executable skills	Task and environment feedback	Persistent adaptation; update mechanism need not evolve.
STaR / Self-Instruct	Model parameters	Filtered synthetic supervision	Self-training within a specified pipeline.
Self-Rewarding	Policy and judging behavior	Learned preferences	Coupled improvement claims require external calibration.
AZR / R-Zero	Policy and curriculum	Execution / answer consensus	Adaptive curriculum; grounding differs substantially.
ADAS / GEPA	Agent code / prompts	Task evaluations and traces	Design automation with an outer search procedure.
STOP / Gödel Agent	Improver or agent logic	Downstream utility	Direct bounded self-reference; resources and judge remain specified.
SICA / DGM	Self-modifying scaffold	Coding benchmarks	Inherited code changes; base models and some outer logic remain fixed.
HGM	Descendant population	Lineage-based estimates	Measures productive ancestry; outer search is an authored method.
SEAL	Self-edit policy and adapted weights	Post-update task performance	Learns adaptation choices inside a prescribed outer loop.
FunSearch / AlphaEvolve	Candidate programs	Executable evaluators	Discovery can aid AI development; inheritance must be demonstrated.
AI Scientist	Research artifacts	Experiments and review	Research automation; successor feedback is an additional claim.

Several numerical examples show why the evaluation unit matters. DGM reports an increase from 20.0% to 50.0% on its SWE-bench evaluation subset and from 14.2% to 30.7% on the full Polyglot benchmark. Its SWE-bench results should not be presented as a score on the full official suite. The paper also includes frozen-improver and archive ablations, which are more informative for recursive claims than endpoint scores alone [33].

SICA reports 17% to 53% on a random SWE-bench Verified subset. Its experiment combines a 50-question SWE-bench subset with other tasks, includes time and monetary cost in the utility, and

reports approximately US\$7,000 in API cost for a 15-iteration run [32]. These qualifications identify a bounded experiment, not a general deployment-wide rate of improvement. The SICA and DGM percentages should not be ranked against each other because their evaluation configurations differ.

HGM’s comparison explicitly investigates descendant productivity rather than treating present performance as a sufficient proxy [9]. This supports a shift toward measuring the developmental process. Nevertheless, transfer of a discovered scaffold, prediction of productive lineages, and causal improvement of a reusable operator remain distinguishable results. The proposed protocol in the next section is intended to help separate them without discounting their practical value.

The emerging 2026 benchmarks add complementary measurement targets. PAST-Bench compares later fresh-session behavior with access to retained experience enabled or disabled, and examines evidence of the persistence pathway [40]. RSIBench-Data fixes the surrounding post-training stack while agents revise training-data strategies. It reports improvement over the first valid attempt in 58.33% of settings; among searches continuing after their observed peak, 78.26% finish below that peak [41]. The latter statistic describes regression relative to the within-run maximum in a selected set of runs. It is not the probability that one additional research iteration harms a randomly chosen system.

PostTrainBench allows agents to undertake post-training within a bounded GPU budget and documents both successful targeted adaptation and failures involving benchmark or resource boundaries [42]. AI4AI-Bench separates an agent’s four-hour development period from a subsequent clean-start run of up to twelve hours, using a hidden final evaluator across ten research repositories [43]. These benchmarks make parts of the AI-development process testable. Neither a data-research benchmark nor an algorithm-design benchmark, by itself, establishes that the trained successor repeatedly becomes a better researcher.

6 Evaluating the capacity to improve

6.1 What existing benchmarks measure

SWE-bench measures repository-level issue resolution; MLE-bench measures machine-learning engineering; RE-Bench examines research-engineering performance with comparisons to human experts [44–46]. They can serve as useful operational outcomes, yet a single score on any of them is not an intrinsic measure of recursive improvement. A benchmark that tests the products of research may never test the next researcher’s ability to improve them further.

PAST-Bench addresses experience retention, RSIBench-Data addresses training-data decisions, PostTrainBench addresses autonomous post-training, and AI4AI-Bench emphasizes algorithmic changes [40–43]. Together they motivate a modular evaluation suite rather than one universal RSI number. The relevant modules are persistence, productive proposal, valid selection, inheritance, and multi-generation transfer.

6.2 A matched-operator comparison

To evaluate an improver, let A be a starting target system, U an improvement operator, b a development budget, and V^* an isolated audit evaluator. Define a proposed measure of **improvement yield**:

$$Y(U, A; b) = \mathbb{E} [V^*(\text{Run}(U, A; b)) - V^*(A)].$$

The expectation is over task draws and relevant randomness. Audit inference budgets and environment versions are fixed separately. An updated operator U_t can then be compared with a baseline operator U_0 on the same distribution of starting systems:

$$\Delta_{\text{op}}(b) = \mathbb{E}_{A \sim \mathcal{A}_{\text{audit}}} [Y(U_t, A; b) - Y(U_0, A; b)].$$

A positive estimate supports an operator advantage in the specified setting. It does not establish universal superiority. Matching A matters because a newer operator may otherwise be evaluated only on a more capable target, or on a target with different remaining headroom. Likewise, the operator’s development cost must be charged when comparing complete methods: conditional reuse of an already learned operator and end-to-end discovery of that operator answer different economic questions.

The comparison is a synthesis of existing concerns about metaproductivity, controlled persistence, and adaptive data analysis [9, 40, 47]. It is not a new theorem, and we have not demonstrated that this particular measure predicts long-run general progress.

6.3 Factorial controls and coupled systems

When the target and improver can be separated, a two-by-two experiment helps distinguish their contributions:

Starting target	Original operator U_0	Updated operator U_t
Original target A_0	Baseline development run	Operator effect from the original target
Updated target A_t	Target-state effect under fixed improvement	Joint effect and possible interaction

Every cell uses the same declared development budget, audit protocol, and task split. Comparing columns within a row estimates an operator effect conditional on that target. Comparing rows changes both capability and headroom, so it requires more caution. The factorial design also reveals whether a new operator works only with its own evolved target.

Some systems are tightly coupled: changing the model changes both the solver and the improver, and no clean transplant preserves functionality. In that case, forcing a decomposition would create an artificial system. Researchers should instead compare full lineages against frozen-update controls, identify the intervention precisely, and report the unresolved attribution. Functional self-reference is a property of a system; experimentally isolating its causal contribution is an additional challenge.

Useful controls include a frozen improver given an equal candidate-generation budget, a replay or random-search controller with the same evaluation budget, and a no-persistence condition that removes learned state without changing the task prompt. Equal inference-time sampling is especially important: an evolved agent that simply makes more model calls may improve raw accuracy without improving efficiency. Such a result is still useful, but the claimed resource tradeoff should be explicit.

6.4 Separating development feedback from audit evidence

A robust protocol distinguishes development tasks, selection feedback, and final audit tasks. Candidate search can repeatedly inspect development feedback. Selection procedures can use a separately specified budget of validation feedback. The final audit should assess a frozen candidate or frozen lineage without returning task-specific information that guides further selection. Repeatedly consulting a supposed holdout makes it part of an adaptive optimization process [47].

For long-running systems, one untouched holdout is not enough forever. Successive external audits need fresh task families, controlled disclosure, or a statistical procedure explicitly designed for adaptive reuse. Secret tests reduce direct memorization but do not automatically ensure construct validity. EvalPlus shows the importance of stronger test coverage for evaluating generated code; similarly, an RSI evaluator must check whether the test captures the claimed improvement rather than merely a convenient proxy [48].

Operator transfer deserves its own split. If an improver is developed on one group of codebases, it should be tested on new starting agents or repositories and, where relevant, on another model family. A stronger test asks whether the transplanted improver can generate useful descendants in the new setting. Testing only the transferred agent’s immediate task performance answers a weaker question.

6.5 Lineages, selection, and statistical uncertainty

A self-improvement experiment should retain the entire ancestry graph: candidate hashes, parent identifiers, proposed edits, evaluation access, rejected changes, crashes, and selection decisions. The lineage is the unit that reveals how a modification affected future possibilities. Reporting only the final best agent erases failed branches and makes it difficult to distinguish a productive research procedure from an unusually fortunate draw.

Three curves should be separated: the current deployed candidate, the best candidate selected using permitted validation information, and the retrospective best under audit scores. The last is an analysis artifact, not a deployable selection policy, unless the audit information was available by design and charged to development. A best-so-far curve is non-decreasing by construction; it does not demonstrate that successive modifications are reliably beneficial.

Uncertainty should reflect the level at which treatment was assigned. Task outcomes within one evolved lineage share ancestry and selection history, so treating them as fully independent repetitions can exaggerate precision. Independent lineages, task-family resampling, and paired evaluations with common random seeds address different variance sources. Reports should disclose the number of independent runs and the method used for confidence intervals, rather than assigning a universal sufficient number of seeds.

6.6 Costs, persistence, and acceleration

Resource accounting should include proposal-model inference, candidate execution, evaluator calls, model updates, rejected runs, storage or retrieval overhead where material, and human interventions. Wall time, API expenditure, GPU-hours, and inference tokens are different quantities. A method can improve wall time through parallelism while increasing total compute. A paper should report the relevant vector of costs instead of treating one convenient scalar as a universal budget.

Persistence is measured by evaluating a committed system in fresh sessions without replaying privileged development traces. Capability retention requires additional tests on previous tasks and

constraints. These tests prevent a narrow optimization from being described as cumulative progress when it simply trades one capability for another. If a system changes its goals, comparisons under a stable external objective should be retained; a self-chosen easier objective does not constitute improvement on the original objective.

To study acceleration, let C_t be cumulative development cost and P_t an externally measured performance value on a stable or explicitly calibrated task distribution. A simple descriptive increment is

$$g_t = \frac{P_{t+1} - P_t}{C_{t+1} - C_t}.$$

An increase in g_t over a few observations is not proof of persistent acceleration. The pattern may reflect noise, headroom, changed task difficulty, or a favorable branch. A stronger claim requires replicated generations, an appropriate fixed-improver counterfactual, explicit treatment of changing resources, and transfer beyond the selected benchmark. Saturating scores also make this ratio hard to interpret. We therefore recommend reporting cost–performance frontiers and operator–yield curves together with raw trajectories, without extrapolating a finite experiment into an intelligence-explosion forecast.

7 Failure modes and safety boundaries

7.1 Feedback quality and evaluator co-adaptation

No improvement loop can be interpreted independently of its feedback. LLM judges exhibit biases and limitations documented in benchmark evaluation, while optimizing a proxy reward can reduce performance under a separate reference measure [49, 50]. If a proposer and its evaluator share blind spots, repeated agreement can increase apparent confidence without supplying independent evidence of correctness.

This does not establish a universal ordering in which every formal check is better than every human or model judgment. A proof is strong evidence for the formal proposition it proves; the proposition can still omit an important requirement. A compiler checks a narrower property than a comprehensive behavioral test. A domain expert can evaluate concerns not represented in either. The appropriate question is which failure modes the signal detects, at what cost, and with what calibration to the desired outcome.

When the evaluator itself changes, a further identification problem arises. A higher score can result from a more capable candidate, a more permissive judge, or a changed objective. Evaluate new judges against independently labeled or executable cases, including adversarial disagreement cases, and preserve a reference evaluation that the candidate cannot rewrite. Calibration performance should be reported separately from the policy’s score under the new judge.

7.2 Collapse, forgetting, and loss of diversity

Recursive training on model-generated data can lose information about the original data distribution. The model-collapse literature demonstrates this risk under particular training and sampling conditions; work on accumulating real and synthetic data shows that collapse is not an inevitable consequence of any synthetic-data use [51, 52]. The conclusion is conditional: replacement, accumulation, filtering, and grounding produce different dynamics.

In agent evolution, diversity can disappear even without parameter training. Candidate programs can converge on similar prompts, architectures, or habits because the generator repeatedly conditions on the same successful examples. Self-edit search provides a concrete report of homogenization concerns [35]. Archives can preserve alternatives, but textual variety is not the same as behavioral diversity. Evaluation should probe distinct strategies and transfer properties, as well as the number of stored candidates.

Forgetting presents another tradeoff. A change that improves coding speed can damage error recovery; a memory consolidation rule can discard rare but important constraints. Retention and regression tests should therefore be part of acceptance rather than deferred until deployment. Maintaining a recoverable previous version limits operational damage but does not, by itself, make the research process reliably progressive.

7.3 Self-modification and control integrity

If an agent can edit its evaluator, permissions, logs, or budget accounting, it can alter the measurement and control system as well as the intended task solution. Reward-tampering research provides a causal framework for analyzing such incentives, and PostTrainBench documents relevant boundary violations in contemporary automated training experiments [42, 53]. These are concrete reasons to distinguish the modifiable research workspace from the externally enforced evaluation and resource boundary.

For research systems, a defensible default is to version candidate code and models, isolate execution, restrict access to audit assets and credentials, enforce budgets outside candidate code, and preserve rollback artifacts. More capable internal evaluation can still evolve within this arrangement. The restriction protects the interpretation of the experiment as well as the environment in which it runs.

Broader loss-of-control scenarios remain important hypotheses, but the reviewed bounded experiments do not determine their likelihood or timescale. Conversely, failure to demonstrate open-ended acceleration is not evidence that self-modifying systems are harmless. Scientific claims about recursive capability and engineering decisions about containment should use the evidence appropriate to each question.

8 Research priorities

Measure inheritance directly. Automated AI research should report what artifact is inherited by the next researcher, how it is integrated, and whether it improves subsequent research yield. This would connect training-algorithm and data-research benchmarks to multi-generation RSI experiments rather than treating successful training as closure of the loop.

Develop transferable improver tests. Benchmarks should separate the tasks used to learn an improver from the target systems on which it is asked to make new improvements. Transfer should include producing useful descendants, not only solving new tasks with an already evolved agent. Coupled-system cases need carefully specified lineage controls when transplantation is impossible.

Learn evaluation without surrendering measurement. Self-rewarding and evaluator adaptation are attractive because external labels are expensive. Research should test whether a new evaluator detects errors missed by the previous one, including errors the generator is incentivized to exploit. A growing evaluation capability and a fixed audit contract can coexist.

Report the economics of improvement. Local speedups in coding, training kernels, or data

preparation should be related to total development cost. An expensive improver may be worthwhile when reused widely, but its amortization assumptions must be explicit. A gain in one infrastructure component need not translate proportionally into faster overall research.

Study long-horizon regressions and branching. Strong checkpoints can appear early, and later changes may fail to surpass them. Research should compare lineage selection, checkpoint preservation, diversity maintenance, and stopping rules. Rejected proposals are useful data about the search process and should not disappear from public evidence.

Create shared reporting artifacts. A minimum reproducibility record should include system boundaries, candidate and parent versions, model identifiers, task splits, evaluator access, complete budgets, selection rules, independent-run counts, and failure logs. These artifacts would permit more meaningful comparisons even when full model weights or proprietary APIs cannot be released.

9 Limitations of this survey

The selection is weighted toward language models, coding agents, and computational research because these areas provide inspectable artifacts and executable feedback. Robotics, hardware self-design, developmental learning, and older evolutionary-computation literature are not comprehensively covered. The reference set is a curated sample, and conclusions about the absence of stronger evidence are restricted to that sample rather than asserted as a proof of global absence.

Several central sources are preprints, and some claims depend on proprietary models or author-run infrastructure. Metadata checks do not establish result validity. Differences in task subsets, model versions, budgets, and evaluation access prevent a reliable cross-paper effect-size aggregation. Our proposed operator-yield comparisons also require experimental validation, especially for coupled agents and changing task distributions. No new benchmark results, independent replications, or forecasts are presented.

10 Conclusion

RSI is best studied as an explicit relationship between inherited system changes and the process that generates further changes. Existing work provides important mechanisms for persistent learning, self-directed adaptation, self-modifying scaffolds, productive lineage search, and automated AI research. In the selected literature, these mechanisms support bounded forms of self-improvement, while general, sustained, resource-normalized recursive acceleration remains unestablished.

The next empirical step is to connect mechanism, inheritance, and measurement. Matched improver comparisons, isolated audit feedback, complete lineage records, and honest cost accounting can show when an improvement procedure itself becomes more effective. These practices would make both positive results and plateaus scientifically informative, and reduce the distance between evocative claims of automatic evolution and reproducible evidence.

11 Transparency statement

AI assisted retrieval, drafting, translation, and typesetting. No new experiments were conducted. The source register and evidence ledger document the synthesis; the proposed evaluation protocol remains unvalidated.

A A compact reporting template

Item	Information needed to interpret an RSI claim
System boundary	Model, scaffold, memory, data, improver, evaluator; mutable and externally fixed components
Claimed mechanism	Refinement, persistent adaptation, operator improvement, or repeated acceleration
Inheritance	Exact artifact passed to the next generation; whether the successor conducts the next improvement
Controls	Frozen improver, matched starting target, no persistence, equivalent search and inference budgets
Evaluation access	Development, selection, and audit tasks; feedback returned at each stage
Selection	Current, validation-selected, and retrospective-best candidates reported separately
Resources	Proposal, evaluation, training, failed runs, human work, and deployment inference
Uncertainty	Independent lineages, task-family variation, paired design, confidence-interval method
Transfer and retention	New targets and model families; subsequent improvement; old capability regressions
Reproducibility	Versioned artifacts, ancestry graph, logs, seeds, model identifiers, and failure cases

B Operational glossary

Term	Meaning in this survey
Recursive self-improvement	Inherited change to a mechanism used for subsequent improvement
Persistent adaptation	A committed change available beyond the original episode
Improvement operator	The procedure that proposes or selects changes to a target system
Agent scaffold or harness	Prompts, tools, memory interfaces, and control flow around a foundation model
Metaproductivity	Capacity to produce useful descendants; interpreted relative to a task and resource setting
Audit evaluator	An evaluation channel isolated from the candidate’s modification and selection process
Grounding	Evidence or constraints supplied by an environment, specification, or independent reference

References

- [1] Irving John Good. Speculations Concerning the First Ultraintelligent Machine. In *Advances in Computers*, volume 6, pages 31–88. Academic Press, 1965. doi: 10.1016/S0065-2458(08)60418-0. URL <https://www.sciencedirect.com/science/article/pii/S0065245808604180>.
- [2] Juergen Schmidhuber. Goedel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements, 2003. URL <https://arxiv.org/abs/cs/0309048>. arXiv preprint; version 5 consulted.
- [3] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, et al. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence, 2025. URL <https://arxiv.org/abs/2507.21046>. arXiv preprint; version 4 consulted.
- [4] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, et al. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems, 2025. URL <https://arxiv.org/abs/2508.07407>. arXiv preprint; version 2 consulted.
- [5] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops, 2026. URL <https://arxiv.org/abs/2607.07663>. arXiv preprint; version 1 consulted.
- [6] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, et al. Self-Improvements in Modern Agentic Systems: A Survey, 2026. URL <https://arxiv.org/abs/2607.13104>. arXiv preprint; version 1 consulted.
- [7] Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, et al. The Path to Recursive Self-Improving Agents: Foundation, Framework, and Future Directions, 2026. URL <https://www.preprints.org/manuscript/202608.0051>. Preprints.org preprint, version 1.
- [8] Hao Zhou, Haichuan Hu, Tianyu Luo, Ye Shang, Chunrong Fang, Zhenyu Chen, Liang Xiao, and Qunjun Zhang. Self-Evolving Coding Agents, 2026. URL <https://arxiv.org/abs/2608.03392>. arXiv preprint; version 3 consulted.
- [9] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine, 2025. URL <https://arxiv.org/abs/2510.21614>. arXiv preprint; version 3 consulted.
- [10] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation, 2023. URL <https://arxiv.org/abs/2310.02304>. arXiv preprint; version 3 consulted.
- [11] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent, 2016. URL <https://arxiv.org/abs/1606.04474>. arXiv preprint; version 2 consulted.
- [12] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving Machine Learning Algorithms From Scratch, 2020. URL <https://arxiv.org/abs/2003.03384>. arXiv preprint; version 2 consulted.

- [13] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>. arXiv preprint; version 1 consulted.
- [14] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions, 2019. URL <https://arxiv.org/abs/1901.01753>. arXiv preprint; version 3 consulted.
- [15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, et al. Self-Refine: Iterative Refinement with Self-Feedback, 2023. URL <https://arxiv.org/abs/2303.17651>. arXiv preprint; version 2 consulted.
- [16] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL <https://arxiv.org/abs/2303.11366>. arXiv preprint; version 4 consulted.
- [17] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL <https://arxiv.org/abs/2305.16291>. arXiv preprint; version 2 consulted.
- [18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, 2023. URL <https://arxiv.org/abs/2302.04761>. arXiv preprint; version 1 consulted.
- [19] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet, 2023. URL <https://arxiv.org/abs/2310.01798>. arXiv preprint; version 2 consulted.
- [20] Dancheng Liu, Amir Nassereldine, Ziming Yang, Chenhui Xu, Yuting Hu, Jiajie Li, et al. Large Language Models have Intrinsic Self-Correction Ability, 2024. URL <https://arxiv.org/abs/2406.15673>. arXiv preprint; version 2 consulted.
- [21] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, et al. Training Language Models to Self-Correct via Reinforcement Learning, 2024. URL <https://arxiv.org/abs/2409.12917>. arXiv preprint; version 2 consulted.
- [22] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning With Reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>. arXiv preprint; version 2 consulted.
- [23] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions, 2022. URL <https://arxiv.org/abs/2212.10560>. arXiv preprint; version 2 consulted.
- [24] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-Rewarding Language Models, 2024. URL <https://arxiv.org/abs/2401.10020>. arXiv preprint; version 3 consulted.
- [25] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models, 2024. URL <https://arxiv.org/abs/2401.01335>. arXiv preprint; version 3 consulted.
- [26] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, 2025. URL <https://arxiv.org/abs/2501.12948>. arXiv preprint; version 2 consulted.

- [27] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data, 2025. URL <https://arxiv.org/abs/2505.03335>. arXiv preprint; version 3 consulted.
- [28] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, et al. R-Zero: Self-Evolving Reasoning LLM from Zero Data, 2025. URL <https://arxiv.org/abs/2508.05004>. arXiv preprint; version 4 consulted.
- [29] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems, 2024. URL <https://arxiv.org/abs/2408.08435>. arXiv preprint; version 2 consulted.
- [30] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning, 2025. URL <https://arxiv.org/abs/2507.19457>. arXiv preprint; version 2 consulted.
- [31] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement, 2024. URL <https://arxiv.org/abs/2410.04444>. arXiv preprint; version 4 consulted.
- [32] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A Self-Improving Coding Agent, 2025. URL <https://arxiv.org/abs/2504.15228>. arXiv preprint; version 2 consulted.
- [33] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents, 2025. URL <https://arxiv.org/abs/2505.22954>. arXiv preprint; version 3 consulted.
- [34] Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-Adapting Language Models, 2025. URL <https://arxiv.org/abs/2506.10943>. arXiv preprint; version 2 consulted.
- [35] Alistair Cheong, Haolin Cong, Tyler Yang, and Dustin Miao. Search over Self-Edit Strategies for LLM Adaptation, 2026. URL <https://arxiv.org/abs/2601.14532>. arXiv preprint; version 1 consulted.
- [36] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL <https://www.nature.com/articles/s41586-023-06924-6>. Published online 14 December 2023.
- [37] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>. arXiv preprint; version 1 consulted.
- [38] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, 2024. URL <https://arxiv.org/abs/2408.06292>. arXiv preprint; version 3 consulted.
- [39] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, 2025. URL <https://arxiv.org/abs/2504.08066>. arXiv preprint; version 1 consulted.
- [40] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, et al. PAST-Bench: Benchmarking the Foundations of Recursive Self-Improvement in Personal Agents, 2026. URL <https://arxiv.org/abs/2608.04003>. arXiv preprint; version 1 consulted.

- [41] Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. RSIBench-Data: Benchmarking Data-Centric Research for Recursive Self-Improvement, 2026. URL <https://arxiv.org/abs/2607.25886>. arXiv preprint; version 1 consulted.
- [42] Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. PostTrainBench: Can LLM Agents Automate LLM Post-Training?, 2026. URL <https://arxiv.org/abs/2603.08640>. arXiv preprint; version 2 consulted.
- [43] Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, et al. AI4AI-Bench: Benchmarking LLM Agents in Algorithmic Design for Recursive Self-Improvement, 2026. URL <https://arxiv.org/abs/2608.20318>. arXiv preprint; version 1 consulted.
- [44] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2023. URL <https://arxiv.org/abs/2310.06770>. arXiv preprint; version 3 consulted.
- [45] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering, 2024. URL <https://arxiv.org/abs/2410.07095>. arXiv preprint; version 6 consulted.
- [46] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024. URL <https://arxiv.org/abs/2411.15114>. arXiv preprint; version 2 consulted.
- [47] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Preserving Statistical Validity in Adaptive Data Analysis, 2014. URL <https://arxiv.org/abs/1411.2664>. arXiv preprint; version 3 consulted.
- [48] Jiawei Liu, Chungqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation, 2023. URL <https://arxiv.org/abs/2305.01210>. arXiv preprint; version 3 consulted.
- [49] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, 2023. URL <https://arxiv.org/abs/2306.05685>. arXiv preprint; version 4 consulted.
- [50] Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization, 2022. URL <https://arxiv.org/abs/2210.10760>. arXiv preprint; version 1 consulted.
- [51] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024. doi: 10.1038/s41586-024-07566-y. URL <https://www.nature.com/articles/s41586-024-07566-y>. Corrected version of record consulted; correction DOI: 10.1038/s41586-025-08905-3.
- [52] Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, et al. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data, 2024. URL <https://arxiv.org/abs/2404.01413>. arXiv preprint; version 2 consulted.
- [53] Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. Reward Tampering Problems and Solutions in Reinforcement Learning: A Causal Influence Diagram Perspective, 2019. URL <https://arxiv.org/abs/1908.04734>. arXiv preprint; version 5 consulted.